

Assignment No 4

Code:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FinTrack Pro Dashboard</title>
  <link rel="stylesheet" href="style.css">
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&
display=swap" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
  <div class="container">
    <header class="header">
      <div>
        <h1>Finantial Dashboard</h1>
        <p id="currentDate"></p>
      </div>
    </header>
    <section class="summary">
      <div class="card balance">
        <h3>Balance</h3>
        <p id="totalBalance">₹0.00</p>
      </div>
      :
      :
      :
    <section class="charts">
      <div class="card chart-box">
        <h3>Income vs Expense</h3>
        <canvas id="barChart"></canvas>
      </div>
      :
      :
      :
    </div>
    <script src="script.js"></script>
  </body>
</html>
```

CSS

```
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}

.summary {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 15px;
  margin-bottom: 20px;
}

.card {
  background: var(--card);
  padding: 20px;
  border-radius: 15px;
  box-shadow: 0 5px 20px rgba(0, 0, 0, 0.1);
  transition: 0.3s;
}

.card:hover {
  transform: translateY(5px);
}

.form-section form {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 10px;
}

.charts {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
  margin-top: 20px;
}

.chart-box canvas {
  height: 250px !important;
}

table {
  width: 100%;
  margin-top: 10px;
  border-collapse: collapse;
}

th,td {
  padding: 12px;
  text-align: left;
```

JAVASCRIPT

```
let transactions = JSON.parse(localStorage.getItem("transactions")) || [];
:
:
let barChart, doughnutChart, lineChart;
const formatCurrency = (amount) => {
  return new Intl.NumberFormat("en-IN", {
    style: "currency",
    currency: "INR"
  }).format(amount);
};
function update() {
  let income = 0;
  let expense = 0;
  list.innerHTML = "";
  transactions.forEach(t => {
    if (t.type === "income") {
      income += t.amount;
    } else {
      expense += t.amount;
    }
  });
  list.innerHTML += `
    <tr>
      <td>${t.date}</td>
      :
      <td>
        <button class="delete" onclick="deleteTx(${t.id})">X</button>
      </td>
    </tr>
  `;
});
balanceEl.innerText = formatCurrency(income - expense);
incomeEl.innerText = formatCurrency(income);
expenseEl.innerText = formatCurrency(expense);
localStorage.setItem("transactions", JSON.stringify(transactions));
updateCharts(income, expense);
}
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const transaction = {
    id: Date.now(),
    description: description.value,
    amount: +amount.value,
    type: type.value,
    category: category.value,
    date: date.value
  };
  transactions.push(transaction);
```

```

    update();
    form.reset();
  });
  function deleteTx(id) {
    transactions = transactions.filter(t => t.id !== id);
    update();
  }
  function updateCharts(income, expense) {
    if (barChart) barChart.destroy();
    barChart = new Chart(ctxBar, {
      type: "bar",
      data: {
        labels: ["Income", "Expense"],
        datasets: [{
          data: [income, expense],
          backgroundColor: ["#06d6a0", "#ef476f"]
        }]
      }
    });
    const categories = {};
    transactions.forEach(t => {
      if (t.type === "expense") {
        categories[t.category] =
          (categories[t.category] || 0) + t.amount;
      }
    });
    if (doughnutChart) doughnutChart.destroy();
    doughnutChart = new Chart(ctxDoughnut, {
      type: "doughnut",
      data: {
        labels: Object.keys(categories),
        datasets: [{
          data: Object.values(categories)
        }]
      }
    });
    let balance = 0;
    const trend = [];
    transactions.forEach(t => {
      balance += t.type === "income"
        ? t.amount
        : -t.amount;
      trend.push(balance);
    });
    if (lineChart) lineChart.destroy();
  }

```

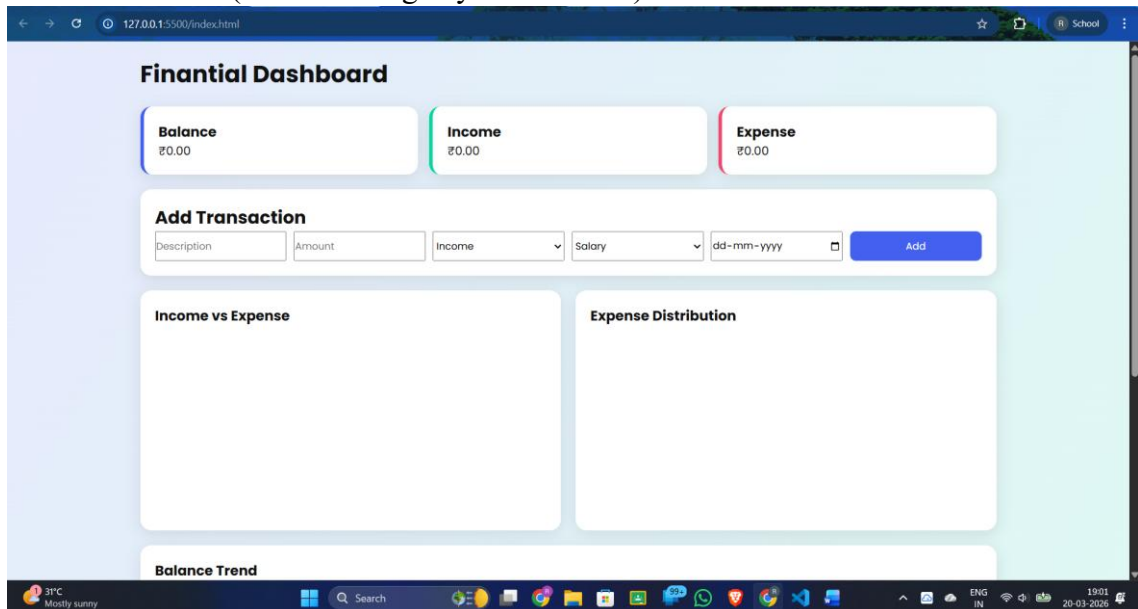
```

lineChart = new Chart(ctxLine, {
  type: "line",
  data: {
    labels: transactions.map(t => t.date),
    datasets: [{
      data: trend,
      borderColor: "#4361ee",
      fill: true
    }]
  }
});
}
});
}
document.getElementById("darkModeToggle").onclick = () => {
  document.body.toggleAttribute("data-theme");
};
document.getElementById("currentDate").innerText =
  new Date().toLocaleDateString();
update();

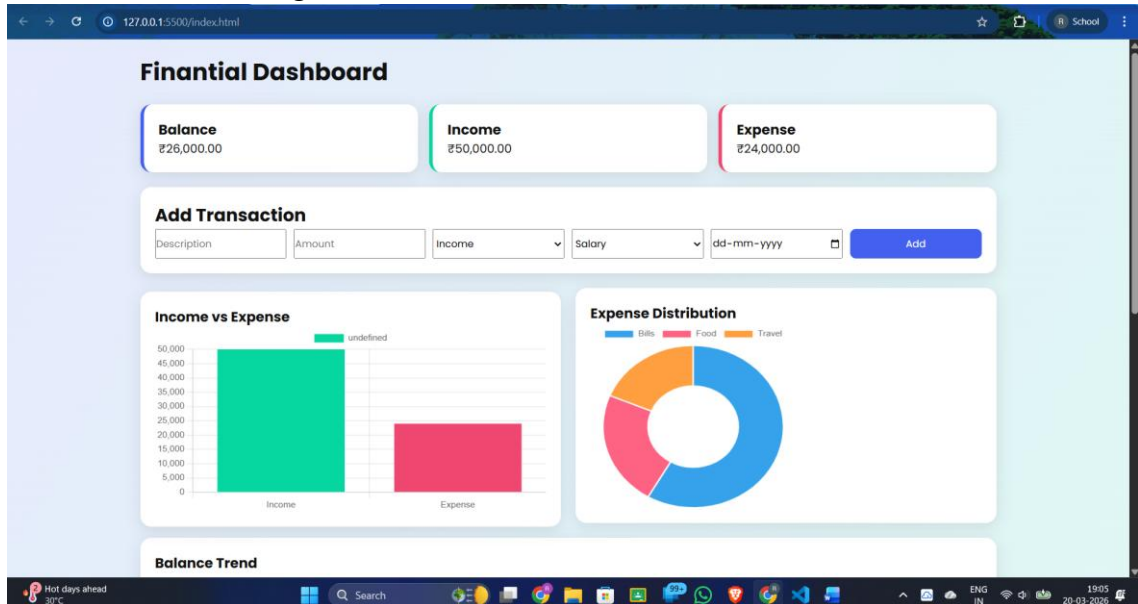
```

Output:

Initial Dashboard(before adding any transactions)



Dashboard after adding financial data



List of all transactions

